

Frugal Router

A Token-Efficient AI Agent

Bisman Singh · Chirag Sharma
AMD Developer Hackathon Act II · Track 1

The whole game is one rule

Pass the accuracy gate, then **win on the fewest Fireworks tokens**.

Every point of accuracy above the bar is money you did not need to spend.

Why this is not a routing problem

The organizers confirmed every scored answer must come from a Fireworks call. So the question is never *whether* to call the API. It is **how cheap each call can be**.

- A free local model does the thinking
- Fireworks does the answering
- Confidence decides how much the remote model has to do

The pipeline

```
/input/tasks.json
-> classify          8 categories, free, on the prompt
-> local model      drafts + measures its own confidence (free)
-> confidence gate  self-consistency + logprob quantile
                    confident -> compact confirmation call (~10 tokens)
                    unsure   -> full remote solve, reasoning only where needed
-> judge-shaped answer
/output/results.json    every task answered, always exit 0
```

Local intelligence is free, so we spend it lavishly. Remote tokens are the score, so we spend them once.

The confidence stack

Small models are the whole difficulty. We kept only the signals that hold up at that scale.

- **Self-consistency voting** with early stop on agreement
- **Pessimistic logprob quantile**, so one unsure token shows through
- **Cut** yes or no self-verification, because small models say yes to almost everything

Confidence sets how hard the remote model has to work, per category.

Token engineering

Measured on a live Fireworks evaluation, not guessed.

- **Reasoning effort capped** cut completion tokens by roughly a third
- **Parallel remote calls** so a large batch clears the ten minute budget
- **Answer contracts** shaped for the intent judge, never bare labels
- **Nothing cached or hardcoded**, every answer generated fresh

Live run, the real submission image

```
$ docker run --rm -e FIREWORKS_API_KEY -e ALLOWED_MODELS \  
  -v ./in:/input -v ./out:/output \  
  bismansinghmadaan/frugal-router:v1  
  
[ {"task_id":"a","answer":"36"},  
  {"task_id":"b","answer":"Negative: the praise for the display is  
    outweighed by the battery dying by noon."},  
  {"task_id":"c","answer":"def add(a, b):\n    return a + b"} ]
```

Math is right, sentiment carries its justification, code runs. Valid file, clean exit.

Built for the harness, fails safe

- Writes a valid results file **after every task**, so a timeout never zeroes the run
- Hard stop with margin, then always exits clean
- Model ids read from the environment, never baked in
- **No credentials in the image**, verified end to end

Results

Local evaluation across all eight categories.

- **100 percent accuracy**, every category
- **Under 200 tokens per task** at that accuracy
- **75 tests**, all offline and deterministic
- **691 MB** linux and amd64 image, live tested against real Fireworks

Gemma, end to end

- Gemma is the preferred remote model for most categories
- Gemma 3 4B is the recommended local drafting model
- One model family carries both the free path and the paid path

Built for the Best Use of Gemma prize as a config choice, not a bolt on.

Frugal Router

Free intelligence, paid answers, minimum tokens.

github.com/Bisman-Singh/frugal-router

Thank you